

**Day-4**

# Agenda -

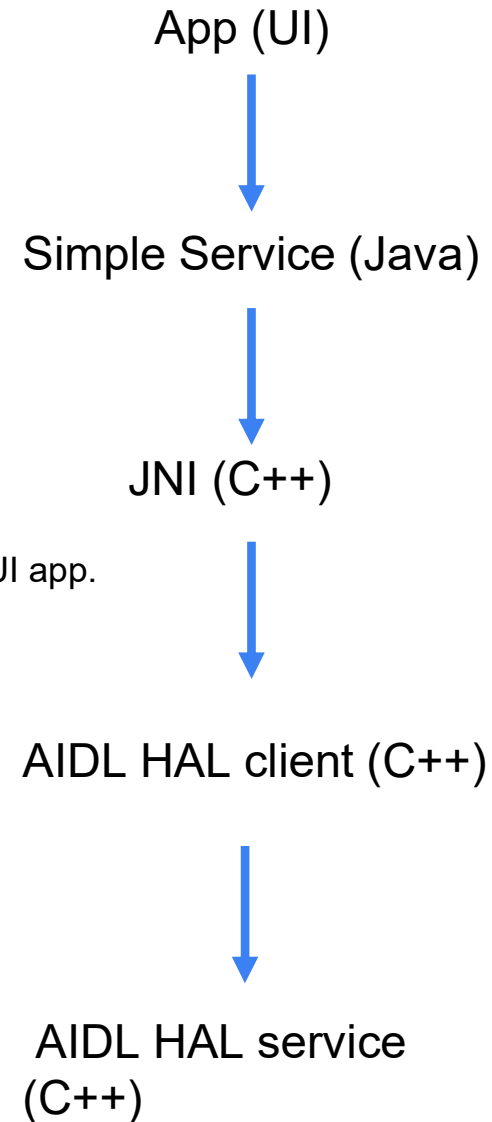
- Using SOME/IP in integrated cockpit environment
  - Service discovery
  - Comm between integrated cockpit ECUs
- Practice -> Android App till HAL Practice
- SELinux and SEPolicy
  - Practice -> SELinux
- Secure booting and secure signing process
- AOSP media player and app development

**Practice -> Android App till HAL Practice**

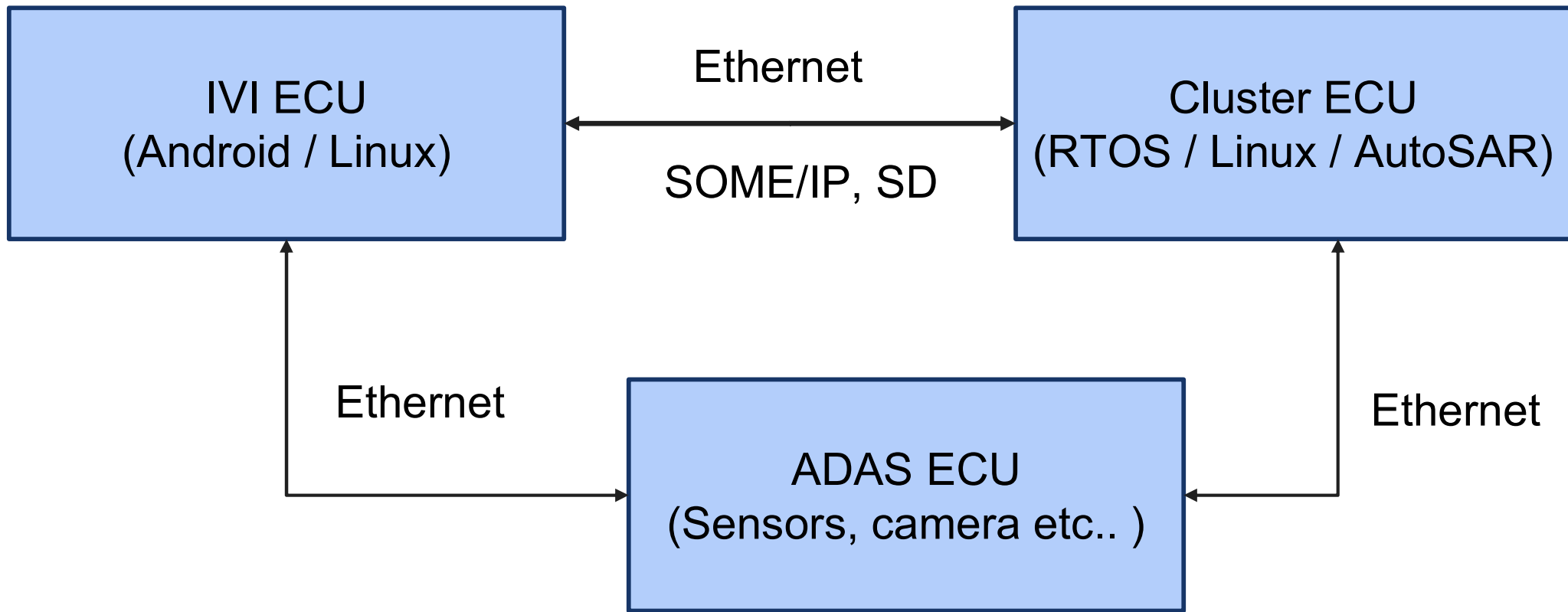
**SOME/IP**

# Current SW Stack Flow

- simple-manager-app
  - Package: com.example.simplemanagerapp
  - Activity: SimpleManagerActivity
  - Shows textbox + buttons (echo, add).
  - Talks to com.example.simpleservice via AIDL (our ISimpleManager, etc.).
- simple-service app
  - Package: com.example.simpleservice
  - Java service (simpleServiceApp / ISimpleServiceImpl) exposes a higher-level API to the UI app.
  - Loads JNI lib (SimpleHalJni.cpp).
  - When it needs the HAL, it calls native methods like nativeAddInts() / nativeEchoString()
    - JNI layer (SimpleHalJni.cpp)
  - Runs inside com.example.simpleservice process.
  - Uses NDK Binder client for vendor.example.simple.ISimple/default.
  - Converts Java types ↔ C++ types and calls the HAL methods.
    - Simple HAL service process
- Native binary: vendor.xrda3.simple-hal-service
- Started at boot by simple-hal-xrda3.rc.
- Implements ISimple using SimpleHal class.
- Registers itself with binder as vendor.example.simple.ISimple/default.



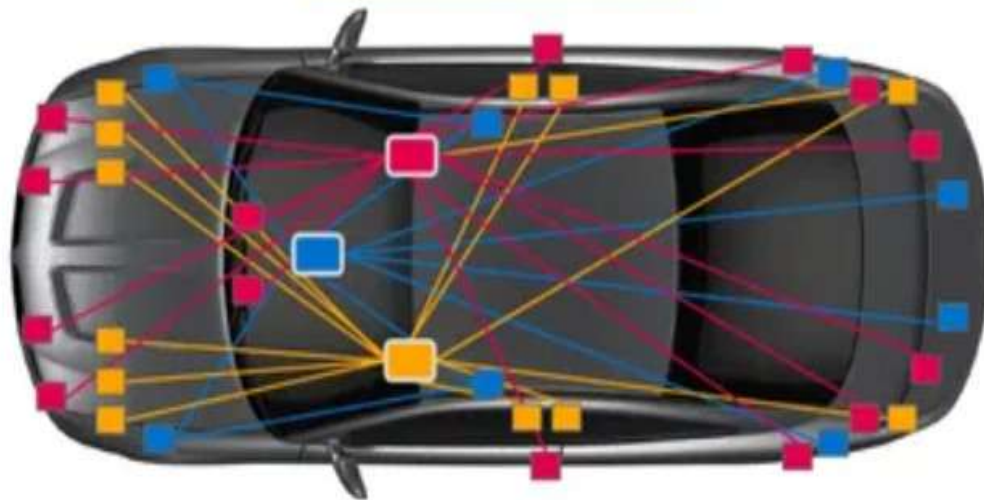
# **Integrated Cockpit Architecture**



integrated cockpit = **IVI + Digital Cluster + Telematics + ADAS + Connectivity Gateway** running on **multiple ECUs**.

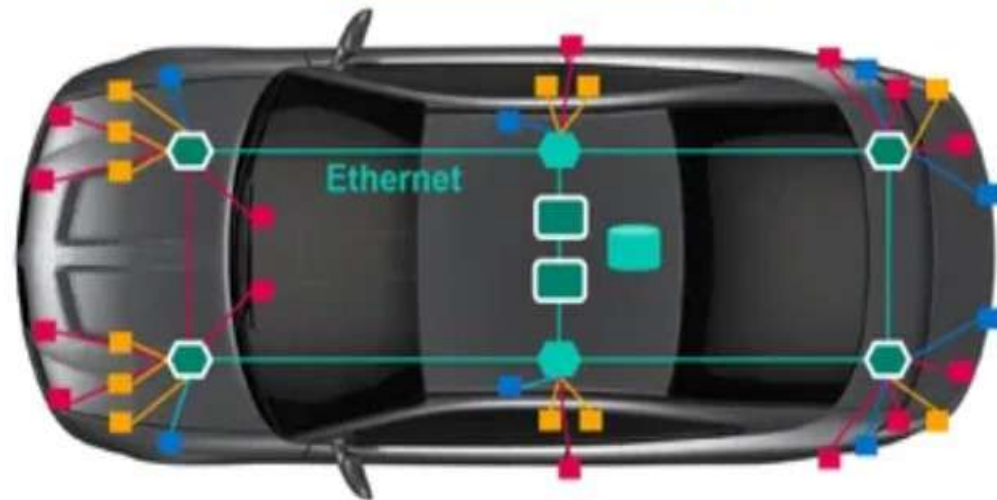
# Software-defined demands zonal architecture

Domains



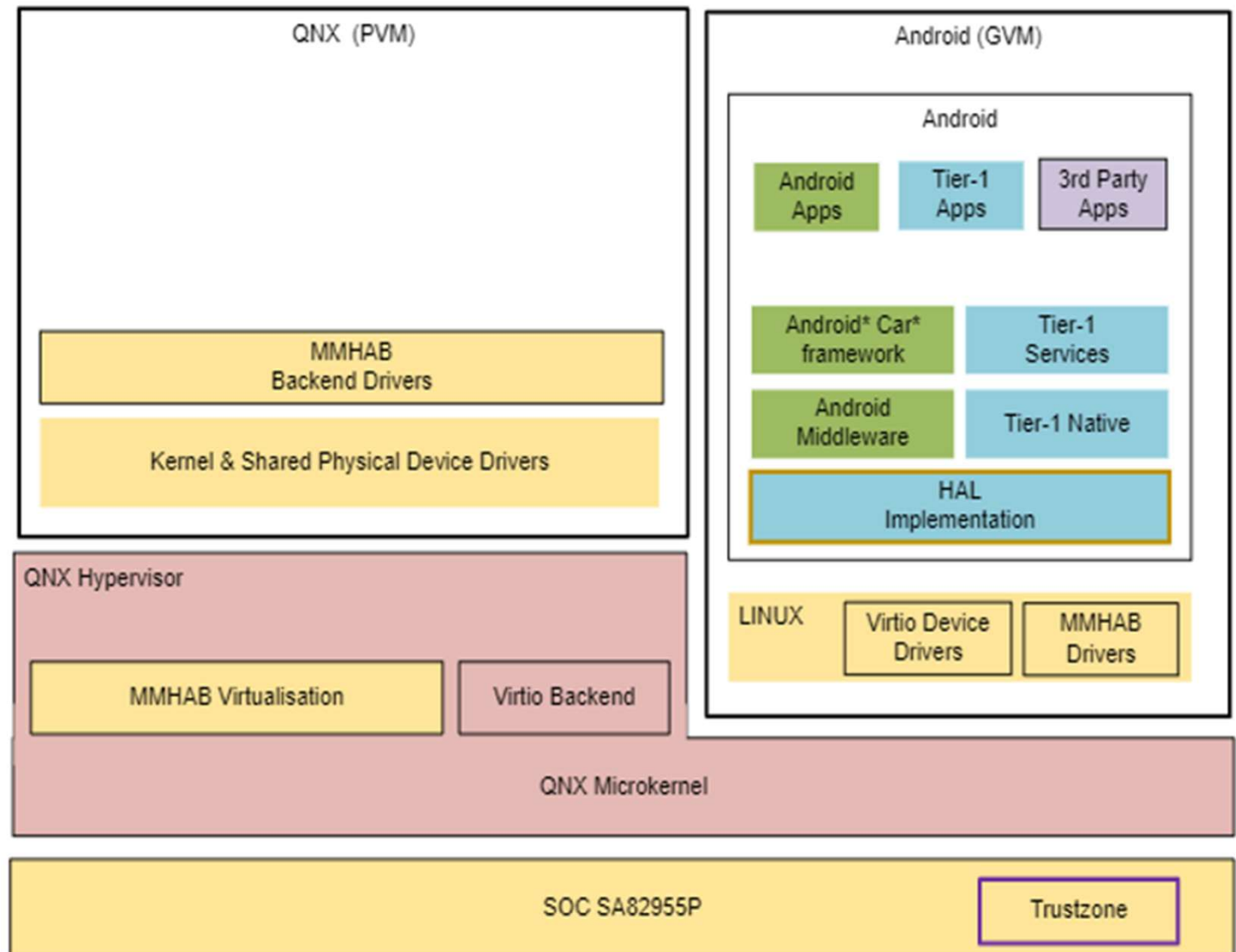
Point to point, rigid, expensive

Zones



Networked, secure, scalable, intelligent

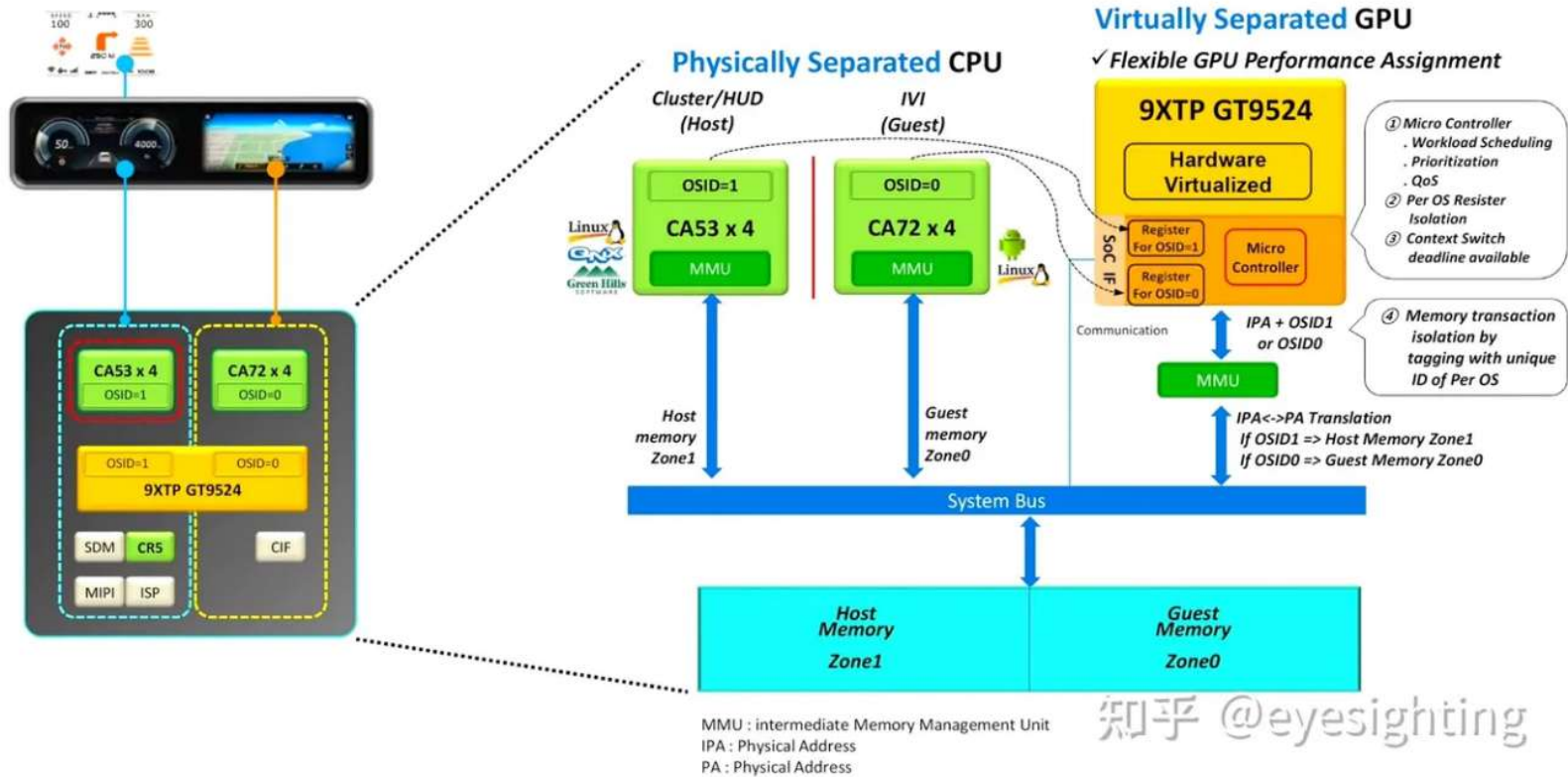
# Integrated Cockpit Architecture Overview



# Telechip Based Solutions

Dolphin3

## Flexible Hypervisor-less Solution (HLS)



知乎 @eyesighting

# What is SOME/IP and Why SOME/IP ?

## Problem -

- Automotive technology is growing rapidly with wider applications to it. For e.g - GPS, real-time traffic updates, V2X connectivity, ADAS, precise telematics systems for in-vehicle navigation etc..
- All of these growing automotive applications are driving the automotive bandwidth requirements.
- Hence it requires to think beyond CAN, LIN, Flexray, and other interfaces

## Solutions -

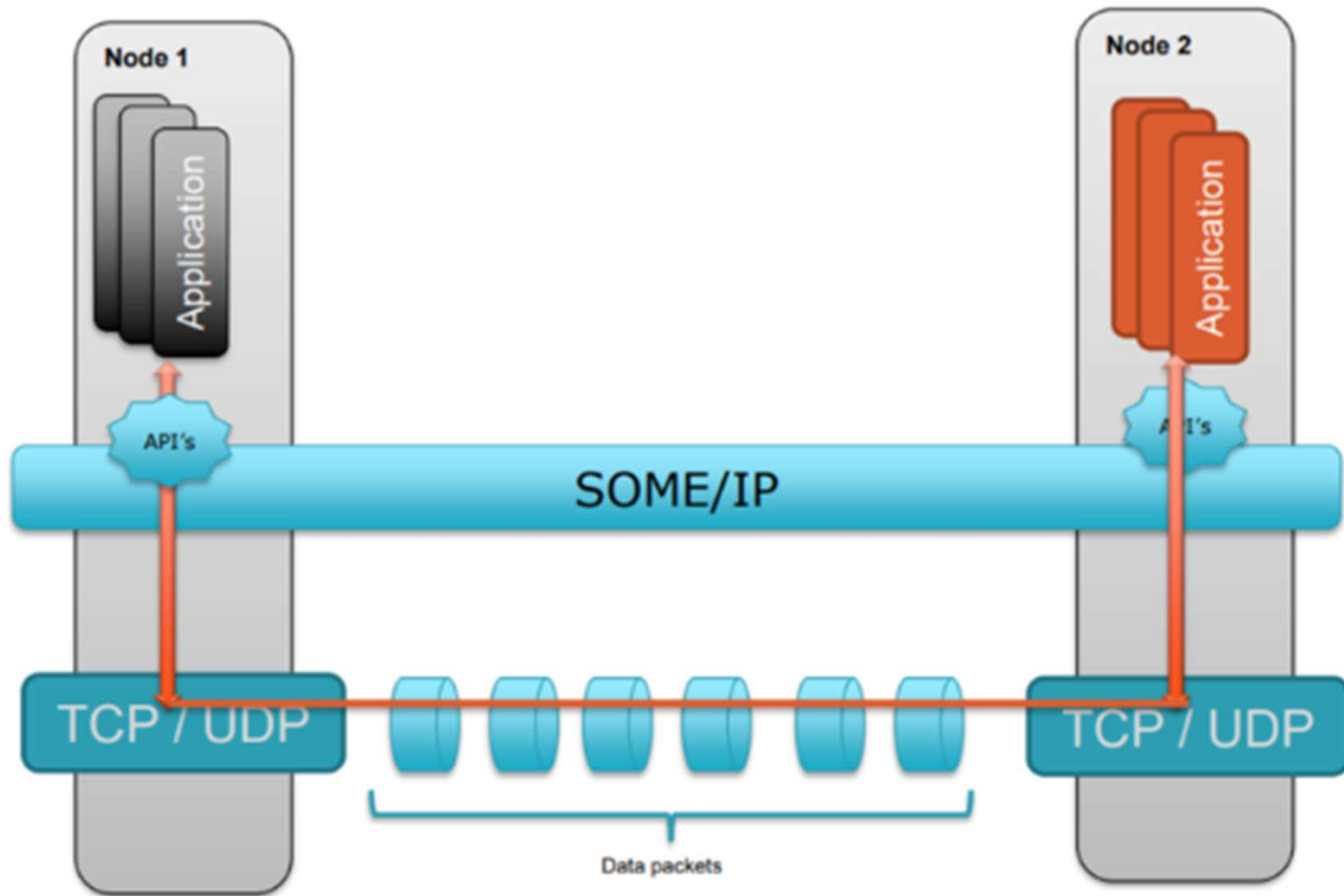
- Ethernet emerged as the best fit considering the changing communication requirements of the ECU within the vehicle.
- With this ethernet needed a middleware solution to transport control data in Automotive.
- The middleware control data communication should: -
  - Support CAN like communication.
  - Support MOST like control communication.
  - Shall efficiently support switched medium.

To address these needs SOME/IP was created.

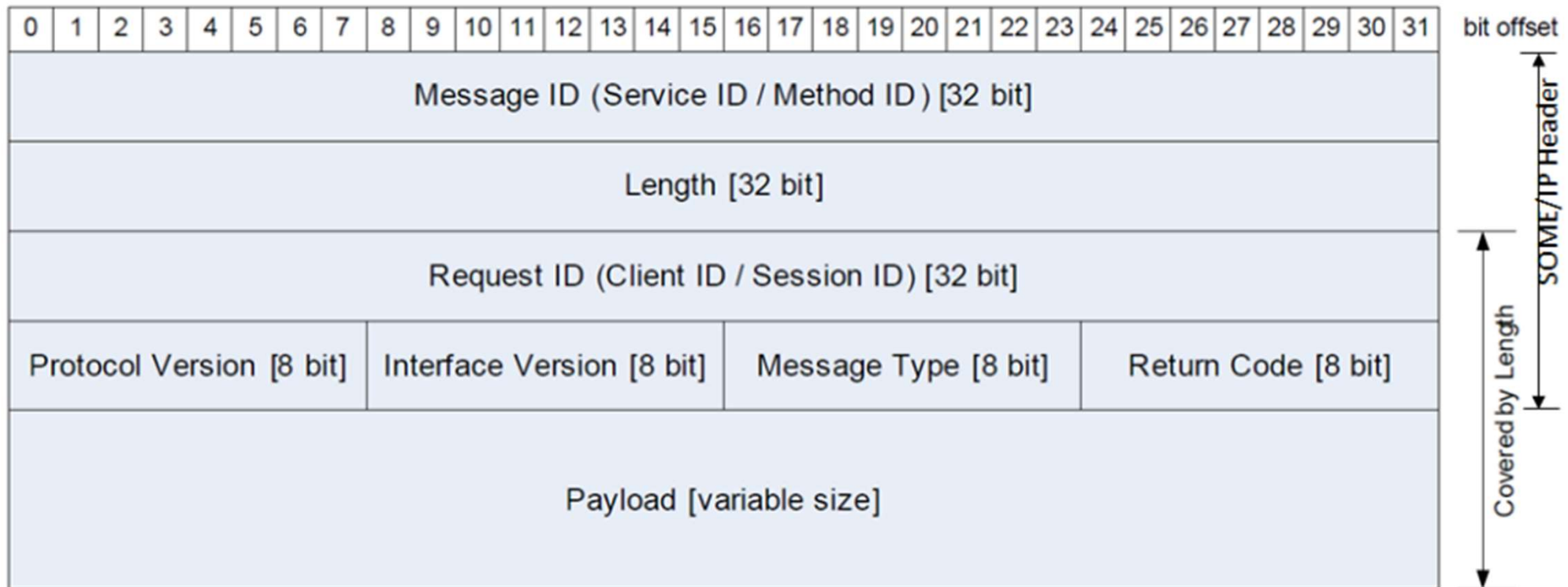
# What is SOME/IP and Why SOME/IP ?

- SOME/IP is an automotive middleware solution that is used for sending control messages over Ethernet.
- SOME/IP full form is - Scalable Service-Oriented Middleware over IP.
- It supports remote procedure calls, event notifications
- It is a service-oriented architecture meaning, a sender only sends data when at least one receiver in the network needs the data.
- The advantage of this procedure is that the network and all connected nodes are not loaded with unnecessary data.

# What is SOME/IP and Why SOME/IP ?



# SOME/IP serialized data header format



# SOME/IP Service Discovery (SD) Overview

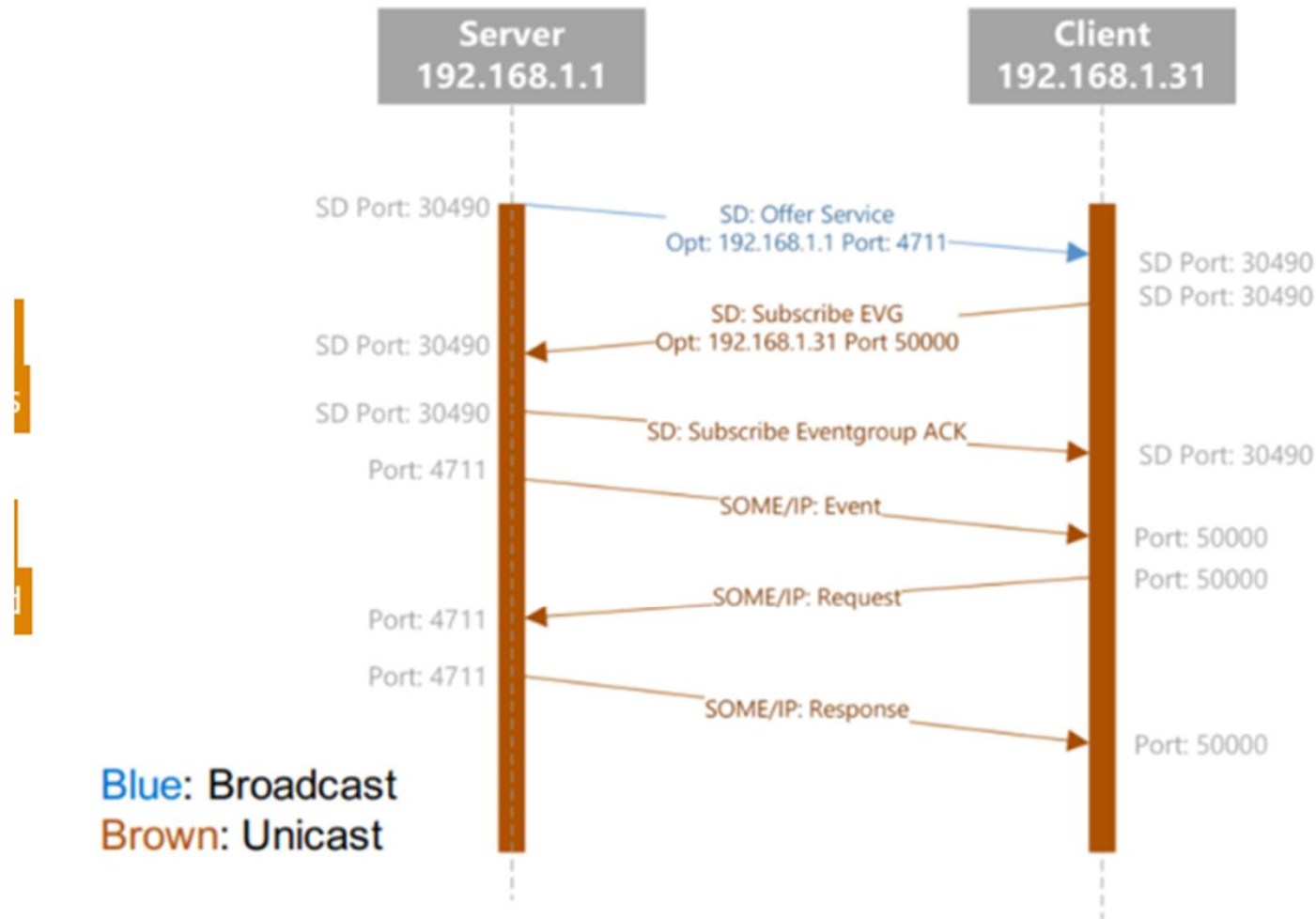
## Why SD -

- In automotive, services come up at different times
- Because Boot-order is NOT fixed
- (IVI takes ~25-30 sec, cluster takes 2-3 sec).
- So SD lets ECUs **automatically find each other**.

## SD Flow

- **Server ECU → OfferService**
- **Client ECU → FindService**
- SD matches Offer + Find
- Client → SubscribeEventGroup
- Server → sends events (Speed, Navigation, Alerts)

# SOME/IP Service Discovery (SD) Overview



# SD Message Types

| SD Message Type             | Purpose                            |
|-----------------------------|------------------------------------|
| <b>OfferService</b>         | Server says "I provide Service X"  |
| <b>StopOfferService</b>     | Server temporarily unavailable     |
| <b>FindService</b>          | Client searching for a provider    |
| <b>Subscribe EventGroup</b> | Client registers for event updates |
| <b>StopSubscribe</b>        | Client no longer needs updates     |

## Example

Cluster → wants Speed

Speed ECU → publishes it

Cluster sends:

```
FindService(ServiceID = SPEED_SERVICE)  
Subscribe(EventGroup = SPEED_EVENTS)
```

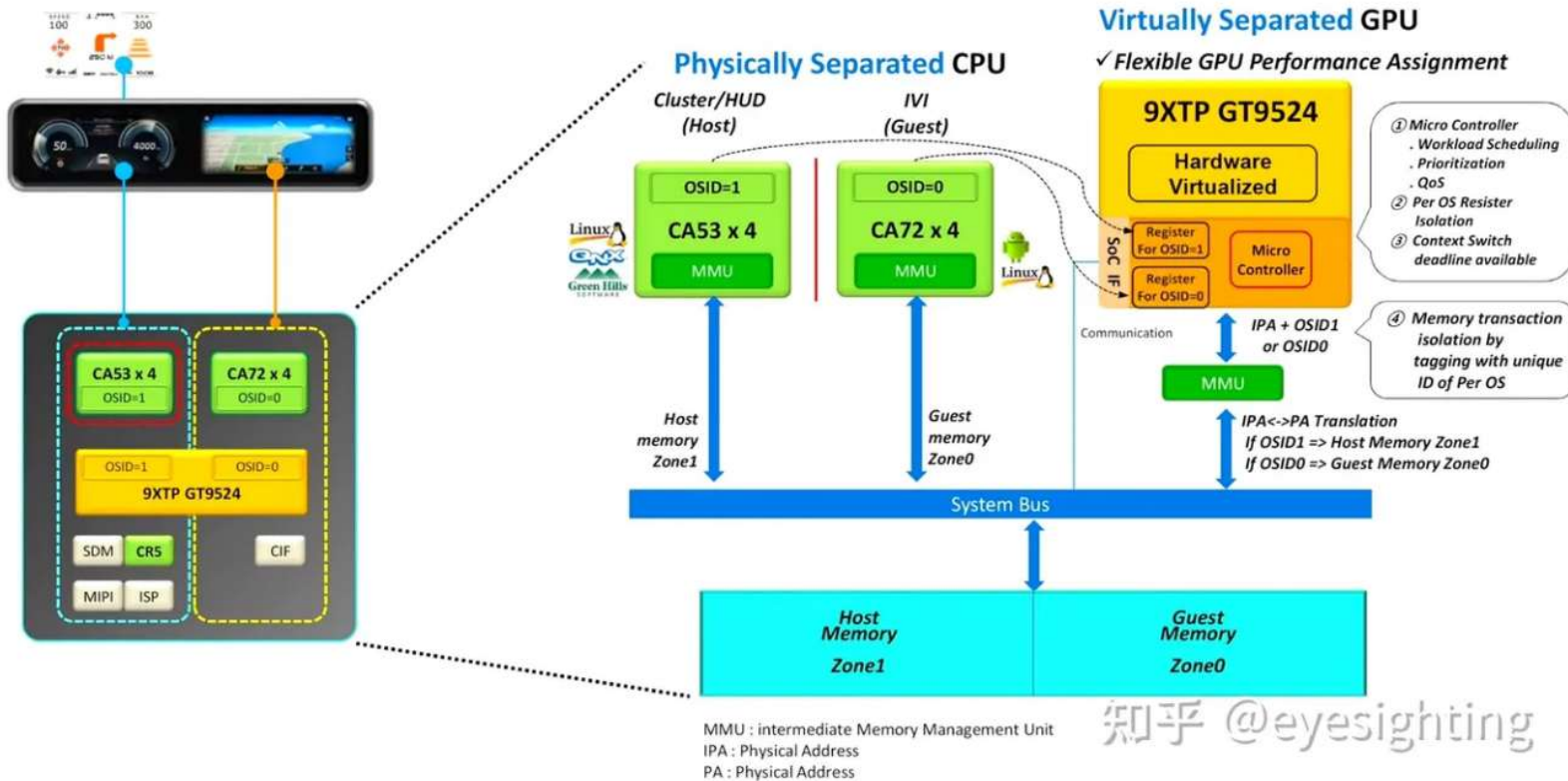
Speed ECU responds:

```
OfferService(ServiceID = SPEED_SERVICE)
```

# Telechip Based Solutions

## Flexible Hypervisor-less Solution (HLS)

Dolphin3



Why Telechips does NOT use SOME/IP or Ethernet inside the SoC?

# SD Message Types

| SD Message Type             | Purpose                            |
|-----------------------------|------------------------------------|
| <b>OfferService</b>         | Server says "I provide Service X"  |
| <b>StopOfferService</b>     | Server temporarily unavailable     |
| <b>FindService</b>          | Client searching for a provider    |
| <b>Subscribe EventGroup</b> | Client registers for event updates |
| <b>StopSubscribe</b>        | Client no longer needs updates     |

## Example

Cluster → wants Speed

Speed ECU → publishes it

Cluster sends:

```
FindService(ServiceID = SPEED_SERVICE)  
Subscribe(EventGroup = SPEED_EVENTS)
```

Speed ECU responds:

```
OfferService(ServiceID = SPEED_SERVICE)
```

# **SELinux and SEPolicy**

## SELinux and SEPolicy Introduction

- Android uses SELinux to enforce "the principle of least privilege"
- SELinux can limit access to resources such as files and sockets
  - Even root (Superuser) can be denied access
- It enforces **policies** and It restrict what processes can do
- Each process (native daemon, Android application) operates within a domain
- Any process can access resources only there if there is a rule that allows it
- Because of this it's difficult to exploit a vulnerability because access is limited to that one domain

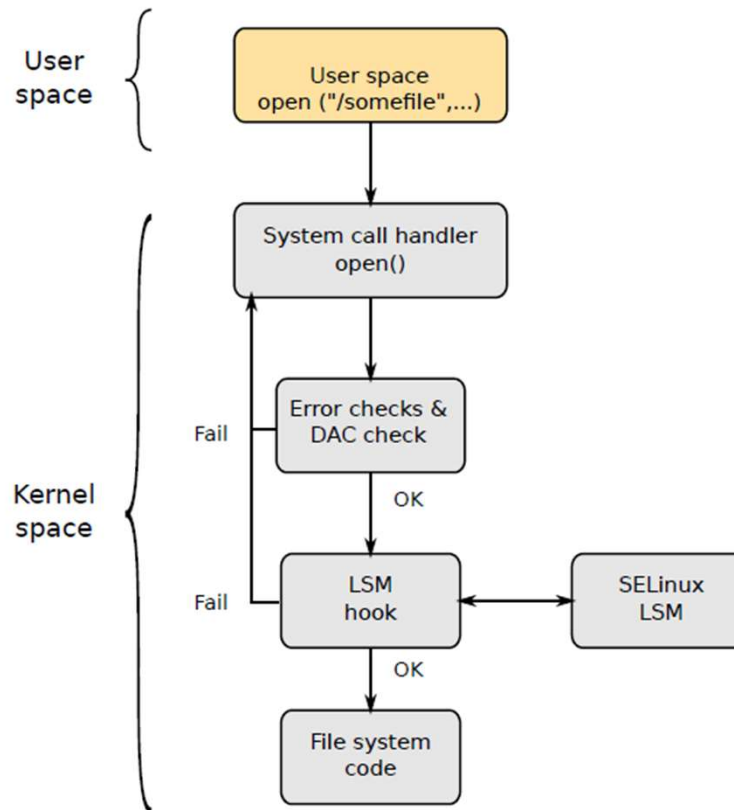
## Why SELinux is Important in Android

- Android runs multiple system services + apps in separate processes
- Each process has a SELinux domain (e.g., system\_app, hal\_audio, vold)
- It Prevents:
  - App -> System file access
  - HAL -> restricted resources
  - Daemon -> unauthorized binder calls
- So it ensures blockage of any unauthorised access.

## DAC and MAC

- Linux has two forms of access control
- DAC (Discretionary Access Control): access to resources is controlled by file modes, UIDs and GIDs
  - This is the default
- MAC (Mandatory Access Control): *additional* restrictions imposed on access to resources
  - Enforced by a Linux security module (LSM), e.g. SELinux
  - Access policy is controlled by user space
  - *MAC cannot grant any access that DAC would deny*: it is additive to DAC

# Linux Security Modules



Android uses SELinux: there are other LSMs, including AppArmor, TOMOYO and Smack

## Android and SELinux

- Permissive mode JB/4.3
  - log illegal access but do not prevent it
- Partial enforcing mode KK/4.4
  - only installd, netd, vold and zygote use enforcing mode, the remainder are still permissive
- Full enforcing mode L/5.0
  - all daemons and apps in enforcing mode

## Context and policy

- SELinux consists of labels and rules
- Processes and files are marked with an SELinux **label**, also known as a **context**
- Which processes can access which files is controlled by a set of **rules**, also known as the **policy**

# SELinux

- SELinux label has 4 parts:  
**user:role:type:level**
- Example:  
`u:r:system_app:s0`
- Most important for Android => **type** (domain)
  - Process has a **domain type**
  - File/resource has a **file type**
- Policy decides which domain can access which file type

## SELinux - Android SEPolicy Structure

### Important folders in AOSP:

| Path                                                         | Purpose                          |
|--------------------------------------------------------------|----------------------------------|
| <code>system/sepolicy/public/</code>                         | Common public policy used by all |
| <code>system/sepolicy/private/</code>                        | Internal Google policy           |
| <code>system/sepolicy/vendor/</code>                         | Vendor-specific rules            |
| <code>device/&lt;vendor&gt;/&lt;product&gt;/sepolicy/</code> | Your device-level custom rules   |
| <code>system/etc/selinux/plat_sepolicy.cil</code>            | Compiled platform policy         |

# SELinux - Policy Components

## **1.Type Enforcement (.te)**

Defines access rules

## **2.File\_contexts**

Maps file paths with SELinux labels

## **3.Service\_contexts**

Maps service names with domains

## **4.Genfs\_contexts**

For pseudo filesystems (proc, sysfs)

## **5.Property\_contexts**

Labels Android system properties

## SELinux - How SELinux Enforces Rules

### Flow:

1. Process starts it gets a SELinux domain label
2. When accessing a file/service:
  - Kernel checks **domain** vs **file type**
3. If no `allow` rule then it **denied** the access.
4. AVC message logged:

```
avc: denied { read } for name="config"  
scontext=u:r:hal_camera:s0  
tcontext=u:object_r:system_file:s0 tclass=file
```

## "Allow" rules

- Suppose demo needs to read files in `/vendor/etc/demo`

Create type `demo_data_file` and give demo read access `demo.te`

```
type demo_data_file, file_type, vendor_file_type;  
allow demo demo_data_file:file { getattr open read ioctl lock map };
```

# Rules

- General form of a rule is

```
rule source-type target-type : class permissions
```

- For example

```
allow demo demo_data_file:file { getattr open read ioctl lock map };
```

- rule = allow
- source-type = demo
- target-type = demo\_data\_file
- class = file
- permissions = {...}

## "dontaudit" rules

- Don't log an avc for this event, but deny in enforcing mode
- dontaudit rules stop the log being swamped with non fatal messages
- Useful if a program is testing for the existence of a resource (file) but handles the error cleanly

Example: system/sepolicy/public/netd.te

```
# Acquire advisory lock on /system/etc/xtables.lock. If this file doesn't  
# exist, suppress the denial.  
allow netd system_file:file lock;  
dontaudit netd system_file:dir write;
```

## "dontaudit" rules

- Don't log an avc for this event, but deny in enforcing mode
- dontaudit rules stop the log being swamped with non fatal messages
- Useful if a program is testing for the existence of a resource (file) but handles the error cleanly

Example: system/sepolicy/public/netd.te

```
# Acquire advisory lock on /system/etc/xtables.lock. If this file doesn't  
# exist, suppress the denial.  
allow netd system_file:file lock;  
dontaudit netd system_file:dir write;
```

## "neverallow" rules

- A neverallow rule cannot be overridden by a later allow rule
- They are used in system/sepolicy to prevent device policy allowing access to protected system resources
- Generally they are not useful when writing device policy

Example `system/sepolicy/public/vendor_init.te`

```
# Init should not be creating subdirectories in /data/local/tmp
neverallow vendor_init shell_data_file:dir { write add_name remove_name };
```

## audit2allow

- `audit2allow(*)` is an SELinux tool that converts AVC audit logs into allow policy rules
- Quick (and lazy) way to generate SELinux policy

```
$ adb pull /sys/fs/selinux/policy
$ adb logcat -b all -d | audit2allow -p policy
#===== httpd =====
allow httpd httpd_data_file:dir search;
allow httpd httpd_data_file:file { getattr open read };
[...]
```

(\*) Not shipped in AOSP from T/13 onwards. Install using  
\$ sudo apt install polycoreutils-python-utils

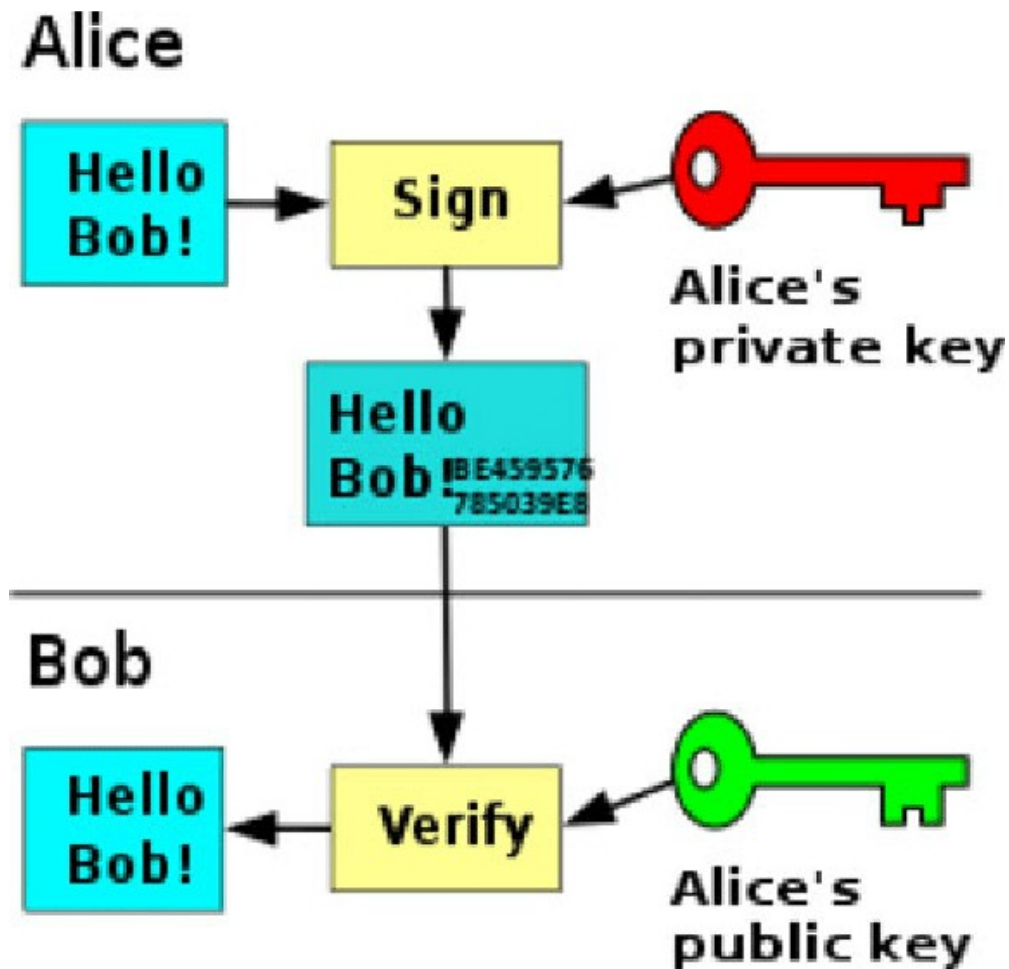
## **SELinux – Practice**

# **Secure booting and secure signing process**

# What is Secure Boot?

- Secure Boot is a security mechanism where the chip verifies every boot-stage firmware using digital signatures.
- If even one image is tampered/modified then boot FAILS.
- This prevents:
  - Malicious firmware
  - Rootkits
  - Modified U-Boot / Kernel
  - Untrusted custom ROMs
  - Malware injection during manufacturing
  - Attacks on boot chain (very common in automotive hacking)
- Telechips TCC805x uses a strict multi-stage secure boot chain.

## Digital Signature (Signing + Verification)



# Why Do We Need Secure Boot in Automotive?

- In automotive, firmware = safety.
- Without secure boot, an attacker could: -
  - Replace bootloader with malicious code
  - Replace MICOM / R5 firmware
  - Modify kernel or initramfs
  - Inject malware through supply chain
  - Dealers / factory / repair shops can unknowingly flash infected firmware.

# Telechips Secure Boot Chain (Firmware Stages)

- Telechips stores **root key** inside **OTP (One-Time Programmable memory)**.
- **OTP guarantees:**
  - Root key cannot be modified
  - Attackers cannot replace keys
  - Hardware enforces authenticity
- **Android / Linux Firmware that must be signed: -**
  - **BL1, BL2, BL3 (both CA72 & CA53)**
  - **SCFW**
  - **HSM firmware**
  - **DRAM params**
  - **OP-TEE**
  - **R5 MICOM firmware**
  - **FWDN image**
  - **Kernel image** (Linux only; Android uses AVB instead)
- Every stage verifies the next stage.

# Telechips Secure Boot Signing Pipeline (How Firmware Is Secured)

- Follow Secure boot steps Demo

# AVB (Android Verified Boot)

- Android Verified Boot (AVB) ensures:
  - System partition
  - Vendor
  - Product
  - Initramfs
- AVB are verified with Google/OEM keys.
- While Telechips Secure Boot verifies boot firmware, AVB verifies Android partitions.
- Together they provide complete chain of trust.

# AVB (Android Verified Boot) Boot Chain

[Boot ROM] → verifies → [Bootloader] (using SoC Secure Boot)

[Bootloader] → verifies → vbmeta

[vbmeta] → verifies → boot.img (kernel+ramdisk)

→ system.img

→ vendor.img

→ product.img

→ odm.img

**If ANYTHING fails → boot stops or enters recovery.**

# What EXACTLY Happens at Boot (Step-by-Step)

- **Step 1 — Bootloader loads vbmeta**
  - Using secure boot (SoC).
- **Step 2 — Bootloader verifies vbmeta**
  - Using AVB public key (OEM-controlled).  
If vbmeta is tampered → FAIL.
- **Step 3 — Bootloader verifies boot.img**
  - Using hash in vbmeta.
- **Step 4 — Bootloader loads kernel with dm-verity parameters**
  - Kernel is told:
  - “system partition must match this root hash”
  - “vendor partition must match this root hash”
- **Step 5 — Kernel mounts partitions with dm-verity**
  - If any block is modified:
  - Kernel triggers VERITY\_ERROR
  - Device enters recovery or reboot loop
- **Step 6 — Android boots only if all partitions match**
  - Therefore, OS cannot be tampered.

# AVB + Secure Boot => Full Chain of Trust

- **Secure Boot (SoC):**
  - Verifies bootloader using hardware keys
- **AVB (Android):**
  - Verifies kernel → system → vendor → product
  - Protects *entire Android filesystem*
- If bootloader is compromised then AVB cannot protect
- If OS is compromised then secure boot prevents entry point compromise

# **AAOS MEDIA PLAYER**

# Build the APK (debug or release)

## From Android Studio (quick):

1. Open your project.
2. Build → Build Bundle(s) / APK(s) → Build APK(s).
3. After build completes, click **Locate** to find the APK (usually [app/build/outputs/apk/debug/app-debug.apk](#) or [release/app-release.apk](#)).

# Prepare the TCC805 device

(These steps are typical for IVI devices; exact menus may vary.)

## 1. Enable Developer Options

- Settings → About → Tap “Build number” (or “Software version”) repeatedly (7 times) until developer mode enabled.

## 2. Enable ADB / USB debugging

- Settings → Developer options → enable **USB debugging** (and **ADB over network** if present).
- Some IVI images have a toggle like “Allow ADB”.

## 3. Grant installs from unknown sources (optional for manual installs)

- Settings → Security → Allow installs from unknown sources (or per-app installer permission). This is required if installing from a file manager.

## 4. Network vs USB

- If the TCC805 exposes ADB over Ethernet/Wi-Fi, note the IP address (Settings → About → Status → IP). You can `adb connect <ip>` instead of USB.
- If connecting via USB, ensure the USB port on device supports ADB (some ports are just for power/media).

# Android Audio Architecture

## Core Components :

- **MediaSession / MediaController** : Standard android media control interface.
- **CarAudioService** : Automotive extension managing zones, focus, routing, and volume groups.
- **CarAudioManager** : App-Facing API to interact with car audio contexts.
- **Audio HAL / AudioControl HAL** : Vendor layer handling hardware routing, gain, fade, balance
- **CAP(Configuration Audio Policy)** : Defines routing and volume rules via XML or Policy Config

# Source Switching

1. User taps Local or USB
2. `releasePlayer()` → frees `MediaPlayer`
3. `initLocalPlayer()` or `initUsbPlayer()`
4. Re-request `AudioFocus`
5. `startPlayer()` resumes playback

```
private void switchSource(String source) {  
    currentSource = source;  
    tvSource.setText("Source: " + source);  
  
    if ("Local".equals(source)) {  
        initLocalPlayer(currentLocalIndex);  
    } else if ("USB".equals(source)) {  
        initUsbPlayer(currentUsbIndex);  
    }  
    updateUi();  
}
```

# Media Source Detection

## Local:

- res/raw/\*.mp3

## USB:

- Detect removable storage in /storage/\*
- Intent ACTION\_MEDIA\_MOUNTED
- Auto reload when USB plugged

# USB Detection

```
File storageRoot = new File("/storage");  
File[] dirs = storageRoot.listFiles();  
for (File d : dirs) {  
    if (d.isDirectory() && d.canRead() &&  
        !d.getAbsolutePath().contains("emulated")) {  
        return d; // USB path  
    }  
}
```

# Audio Focus Flow

## Events:

- GAIN → resume + full volume
- LOSS\_TRANSIENT → pause
- CAN\_DUCK → volume = 0.3f
- LOSS → full pause + abandonFocus

# Audio Focus – Code

```
case AUDIOFOCUS_LOSS_TRANSIENT_CAN_DUCK:  
    player.setVolume(0.3f, 0.3f);  
    tvFocus.setText("FOCUS: DUCK");  
    break;
```

```
case AUDIOFOCUS_GAIN:  
    player.setVolume(1f, 1f);  
    startPlayer();  
    tvFocus.setText("FOCUS: GAIN");  
    break;
```

# Interruptor App – Audio Events

- Navigation → DUCK → Play → UNDUCK
- Alarm → DUCK → Play → UNDUCK
- Voice Call → STOP → Play → RESUME
- Beep → DUCK → Play → UNDUCK

# Interruptor App – DUCK/UNDUCK

```
private void sendBroadcastToPlayer(String action) {  
    Intent intent = new Intent(action);  
    sendBroadcast(intent);  
}
```

```
sendBroadcast(ACTION_DUCK);  
sendBroadcast(ACTION_UNDUCK);
```

# Navigation

1. Interruptor sends ACTION\_DUCK
2. TTS speaks navigation guidance
3. monitorTTSComplete() monitors speaking
4. When done → ACTION\_UNDUCK sent

# Navigation – Code

```
int res = tts.speak(  
    "In 200 meters, take a right.",  
    TextToSpeech.QUEUE_FLUSH, null, "nav"  
);  
  
monitorTTSComplete(() ->  
    sendBroadcastToPlayer(ACTION_UNDUCK)  
);
```

# IVI / Cluster Sounds – Code

- `MediaPlayer mp = MediaPlayer.create(this, R.raw.short_beep);`
- `mp.setOnCompletionListener(m -> m.release());`
- `mp.start();`

# Full Event Flow

**Interruptor App → Broadcast → Player App:**

Navigation:

DUCK → TTS/Audio → UNDUCK

Alarm:

DUCK → raw sound → UNDUCK

Voice Call:

STOP → TTS → RESUME

# **Security Measures in Android Automotive**

Multiuser Management & PIN-based  
Authentication

# Multiuser Management Overview

- Separate profiles for Driver / Guest / Child
- Personalized settings, media, themes
- Stored securely using EncryptedSharedPreferences
- User-specific metadata (theme, PIN, display name)

- Uses Android Security Crypto Library

MasterKey + EncryptedSharedPreferences

- Secure storage for PIN, name, theme

# PIN-based Authentication

- Each user has a unique PIN
- PINs stored encrypted
- Verification done locally
- Authenticated user → Dashboard Activity

## Code : Encrypted Storage

```
MasterKey masterKey = new MasterKey.Builder(ctx)
    .setKeyScheme(MasterKey.KeyScheme.AES256_GCM)
    .build();

prefs = EncryptedSharedPreferences.create(...);
```

## Code : PIN Verification

```
public boolean verifyPin(String userId, String pin) {  
    String stored = prefs.getString(userId + "_pin", null);  
    return stored != null && stored.equals(pin);  
}
```

## User Switching Flow

- LoginActivity verifies PIN
- Loads DashboardActivity with userId
- Theme + preferences loaded per user
- ManageUsersActivity handles add/delete

**Practice - AAOSP App Practice**

**THANK YOU**